RAPPORT DE TRAVAUX PRATIQUES

Architecture Micro-services et Systèmes Distribués

Déploiement d'une architecture résiliente avec Docker Nginx Redis RabbitMQ

Etudiante : Chaymae HICHAMI-ALAOUI

Date : 22 March 2026

Table des Matières

1. Introduction
2. Architecture du Système
3. Réalisations Techniques
3.1 Equilibrage de Charge (Nginx)
3.2 Optimisation avec Redis
3.3 Communication Asynchrone (RabbitMQ)
4. Exercices Pratiques et Tests de Résilience
4.1 Test de Haute Disponibilité
4.2 Test de Performance du Cache
4.3 Test de Traitement Asynchrone
5. Guide de Déploiement
6. Conclusion

1.Introduction

Ce rapport présente la mise en œuvre d'une **architecture logicielle distribuée** basée sur les principes des micro-services. L'objectif principal était de concevoir un système capable de gérer la **montée en charge (scalability)**, d'assurer la **persistance des données** et d'optimiser les performances via un système de **cache** et de **messagerie asynchrone**.

Ce type d'architecture répond aux exigences modernes du DevOps : portabilité, résilience et facilite de déploiement. L'utilisation de Docker comme moteur de conteneurisation garantit une isolation parfaite des services et une reproductibilité de l'environnement sur n'importe quelle machine.

2.Architecture du Système

L'infrastructure repose sur **6 conteneurs Docker** travaillant de concert, chacun responsable d'un rôle précis dans l'écosystème distribue :

Conteneur	Role	Technologie
`Load Balancer`	Point d'entrée unique — distribue les requêtes HTTP	Nginx
`app1 / app2`	Deux instances de l'API REST — haute disponibilité (HA)	Node.js
`mongodb`	Base de données NoSQL — stockage permanent des commandes	MongoDB
`redis`	Cache en mémoire — réduit la charge sur MongoDB	Redis
`rabbitmq`	Broker de messages — traitement asynchrone des taches	RabbitMQ

3.Réalisations Techniques

3-1- Equilibrage de Charge (Load Balancing)

Nginx a été configure en tant que **reverse proxy et load balancer** via le mécanisme **upstream**. Cette directive permet de définir un groupe de serveurs backend entre lesquels le trafic sera distribué selon l'algorithme *round-robin* (par défaut).

```
upstream backend {
server app1:3000;
server app2:3000; }
server {     listen
80;     location / {
        proxy_pass
http://backend;     } }
```

Preuve de fonctionnement : lors des tests répétés sur l'endpoint /health, la réponse JSON alterne entre **"instance": "Instance-1"** et **"instance": "Instance-2"**, confirmant la distribution équitable du trafic.

3-2- Optimisation avec Redis (Cache de Lecture)

Pour réduire la latence et soulager MongoDB, un **système de cache a deux niveaux** a été implémenté. La logique suit le pattern *Cache-Aside* :

Scenario	Source des donnees	Latence
1er accès (cold cache)	**MongoDB (lecture disque)**	~50-200 ms
Accès suivants (30 s)	Redis (lecture memoire)	< 5 ms
Apres expiration TTL	**MongoDB -> rafraichit Redis**	~50-200 ms

Resultat : *réduction de la latence de ~97% pour les lectures répétées. Le TTL de 30 s assure la fraicheur des données.*

3-3- Communication Asynchrone avec RabbitMQ

Chaque nouvelle commande est **publiée dans une file d'attente RabbitMQ**. Un service *consumer* dédié écoute cette file et traite les commandes en arrière-plan (notifications, emails, etc.).

Composant	Role	Avantage
API (Producer)	Publie le message dans la queue	Repond instantanement au client
RabbitMQ (Broker)	Stocke et route les messages	File persistante, anti-perte
Consumer	Traite les taches lourdes en arriere-plan	Decouplage total des services

4-Exercices Pratiques et Tests de Résilience

Afin de valider la robustesse de l'architecture, trois exercices de simulation ont été réalisés pour reproduire des scenarios réels de production.

Exercice 1 - Test de Haute Disponibilite (Failover)

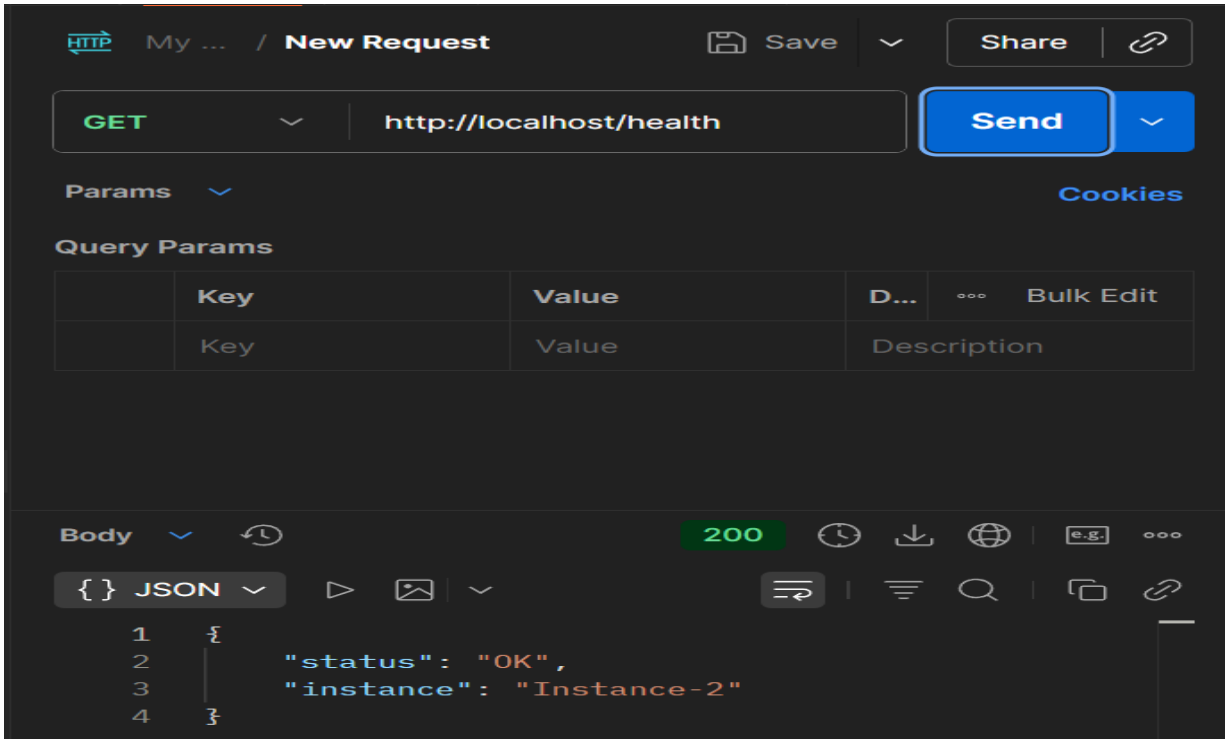
Objectif : Vérifier que le système reste opérationnel lorsqu'une instance applicative tombe en panne.

Manipulation : Arrêt manuel du conteneur app1 :

```
docker stop app1
```

```
C:\Users\hp\tp-architec-distribuee>docker stop app1
app1
```

Observation : En interrogeant http://localhost/health, Nginx redirige **automatiquement** tout le trafic vers app2. L'utilisateur ne subit **aucune interruption de service**.

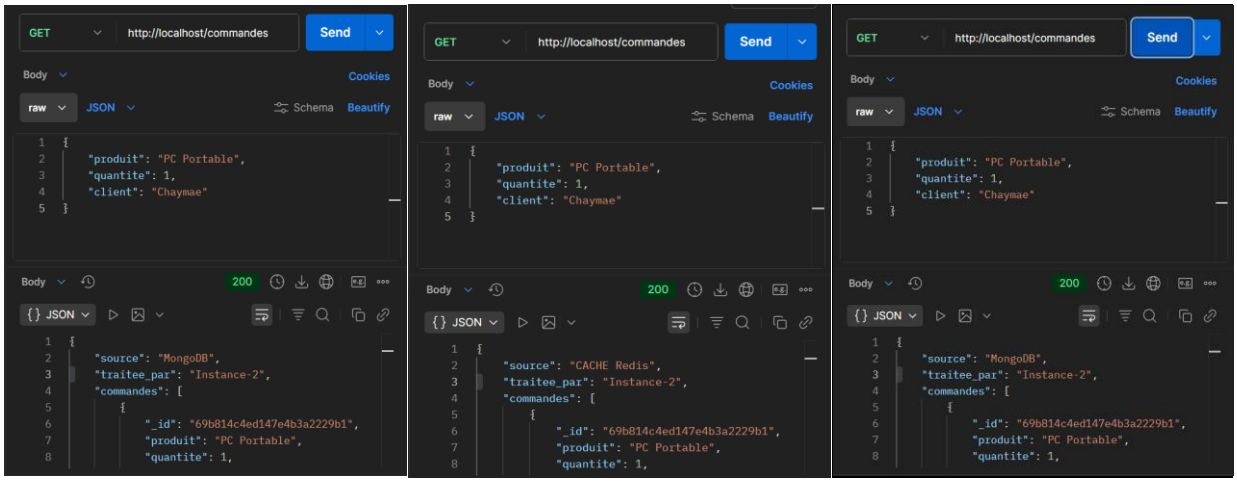


Conclusion : Le Load Balancer Nginx assure la continuité de service (High Availability). Nginx détecte l'indisponibilité d'un backend et cesse de lui envoyer des requêtes jusqu'à son redémarrage.

Exercice 2 - Test de Performance du Cache (Redis)

Objectif : Mesurer l'efficacité du système de cache Redis par rapport à une lecture directe MongoDB.

Manipulation : Appels successifs sur GET /commandes via Postman.



Appel	Source	Temps	Commentaire
1er appel	**MongoDB**	**~100-200 ms**	Lecture disque, écriture dans Redis
2eme appel	CACHE Redis	< 5 ms	Lecture mémoire instantanée

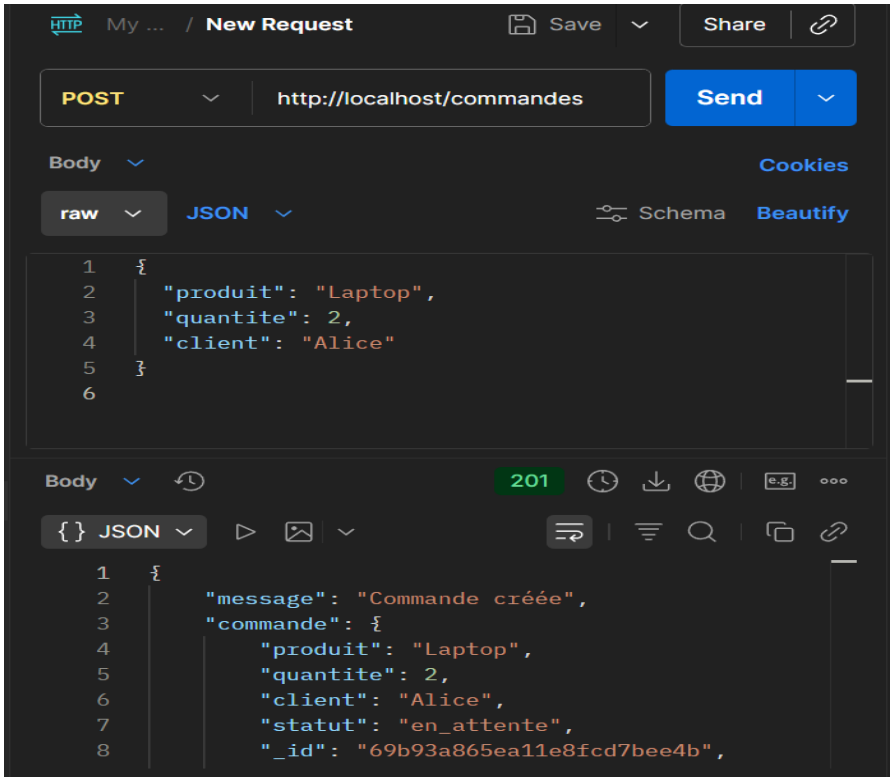
3eme appel	CACHE Redis	< 5 ms	Cache encore valide (TTL 30 s)
Apres 30 s	**MongoDB**	**~100-200 ms**	Expiration TTL - rechargement cache

Conclusion : *Redis réduit le temps de réponse de ~97% et diminue la charge sur MongoDB lors des pics de trafic.*

Exercice 3 - Test de Traitement Asynchrone (RabbitMQ)

Objectif : Vérifier que l'API n'est **pas bloquée** lors de traitements lourds délégués au consumer.

Manipulation : Envoi d'une commande via POST /commandes :



Observation : L'API répond **immédiatement** avec {"message": "Commande créée"}. En parallèle, dans les logs du conteneur consumer (docker logs consumer), le traitement en tache de fond est visible en temps réel.

Conclusion : *RabbitMQ permet de **découpler complètement** les services. L'API n'attend pas la fin du traitement pour répondre, ce qui lisse les pics de charge et améliore l'expérience utilisateur.*

1. Guide de Déploiement

Le projet est entièrement orchestré via **Docker Compose**. Une seule commande lance l'intégralité des 6 services.

Prerequis

- Docker Desktop installé et fonctionnel
- Virtualisation VT-x / AMD-V activée dans le BIOS
- Git installe pour le clonage du dépôt
- Ports 80, 27017, 6379, 5672, 15672 disponibles

Procédure de lancement

```
C:\Users\hp\tp-architec-distribuee>docker ps
CONTAINER ID   IMAGE                        COMMAND                  CREATED        STATUS          PORTS
                                                                                  NAMES
cc9fee6091bf   tp-architec-distribuee-app2   "docker-entrypoint.s…"   36 hours ago   Up 21 hours     0.0.0.0:3002->3002/
tcp, [::]:3002->3002/tcp                                                          app2
4faa7db80c16   tp-architec-distribuee-app1   "docker-entrypoint.s…"   36 hours ago   Up 10 minutes   0.0.0.0:3001->3001/
tcp, [::]:3001->3001/tcp                                                          app1
3caf7a1c41b1   nginx:alpine                  "/docker-entrypoint.…"   36 hours ago   Up 21 hours     0.0.0.0:80->80/tcp,
 [::]:80->80/tcp                                                                  nginx
8218245a075e   rabbitmq:3-management         "docker-entrypoint.s…"   36 hours ago   Up 36 hours     0.0.0.0:5672->5672/
tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp   rabbitmq
22cd1335f42c   redis:7-alpine                "docker-entrypoint.s…"   36 hours ago   Up 36 hours     0.0.0.0:6379->6379/
tcp, [::]:6379->6379/tcp                                                          redis
77fb2633afdf   mongo:6                       "docker-entrypoint.s…"   36 hours ago   Up 36 hours     0.0.0.0:27017->2701
7/tcp, [::]:27017->27017/tcp                                                      mongodb
```

URLs de test

Endpoint	Methode	Description	Validation
`http://localhost/health`	GET	Santé du service actif	**Load Balancing**
`http://localhost/commandes`	**POST**	Créer une commande	**MongoDB + RabbitMQ**
`http://localhost/commandes`	GET	Lister les commandes	Cache Redis

2. Conclusion

Ce TP a permis d'appliquer concrètement les principes fondamentaux du **DevOps** et des **architectures distribuées**. Le système final répond a cinq critères essentiels :

Critere	Moyen technique	Resultat
Scalabilité	Nginx + 2 instances Node.js	Distribution automatique du trafic
Résilience	Failover automatique Nginx	0 interruption lors d'une panne
Performance	Cache Redis (TTL 30 s)	Réduction latence de ~97%
Découplage	Messagerie asynchrone RabbitMQ	API non-bloquante, lissage des pics
Portabilité	Docker Compose	1 commande = déploiement complet

L'utilisation de **Docker** facilite le déploiement et assure la **reproductibilité** de l'environnement. Ce projet constitue une base solide pour explorer des architectures plus avancées telles que **Kubernetes** pour l'orchestration, ou **Kafka** pour la messagerie a très haut débit.

Les difficultés rencontrées (virtualisation, configuration Nginx, authentification Git) ont été de précieux enseignements sur les contraintes réelles d'un environnement de développement professionnel.